Student Result Management System

Bachelor of
Computer Science

Submitted By

AUGUSTINE MONYTOCH MAYAR 31092
MUBARAK ABDULLAZIZ OMAR 108413
VUKONI GODFREY RICHARD 18238
AMOS PETER
PANCHOL AWAN
SUSAN JAMES MONDAY 080055

MARCH 2025



STARFORD INTERNATIONAL UNIVERSITY COLLEGE
''Education for Sustainable Development''
The quality you talk about
Website: thestarford.com
Email:info@starford.com

Table of Contents

1. Introduction 3
2. Scrum Methodology 3
2.1 Scrum Roles 3
2.2 System Backlog 3
2.3 Sprint Schedule 4
2.4 Scrum Board 5
3. User and System Requirements 5
3.1. User Requirements 5
3.2. System Requirements 6
3.2.1. Functional Requirements 6
3.2.2. System Non-Functional Requirements 6
4. System Design 6
4.1. Use Case Diagram 7
4.2 Class Diagram 9
4.3. Activity Diagram 10
4.4. Database Design 10
4.5. Deployment Diagram 11
5. Conclusion 12

Table of Figures

Table 1: Sprint Schedule 1 5
Table 2: Scrum Board 1 5
Table 3: Non - Functional Requirement 1 6
Table 4: Class Attributes 1 9
Table 5: Class Relationships 1 9
Table 6: Database Tables and Attributes 1 11
Table 7: Database Relationships 1 11

Diagrams

Figure 1: Use Case Diagram 1 8
Figure 2: Class Diagram 1 9
Figure 3: Sequence Diagram 1 10
Figure 4: Activity Diagram 1 10

Figure 5: Deployment Diagram 1 11

1. Introduction

This project presents a results management system designed to streamline the processing, management, and retrieval of students' academic results. The system allows students to securely access their examination results by entering their registration numbers and the relevant semester. Lecturers are provided with functionalities to add, update, and manage students' results efficiently. Additionally, the dean serves as the system's super administrator, with full control over user management, including students and lecturers, as well as the ability to edit, delete, and oversee all academic records. The overall design goal of the system is to improve accuracy, accessibility, and administrative efficiency in academic result management.

2. Scrum Methodology

This project will adopt the Scrum methodology, an agile development approach that emphasizes flexibility, collaboration, and iterative progress. Scrum divides the system development process into short, time-boxed iterations known as sprints, during which specific features are designed, developed, and tested. Regular meetings, including sprint planning, daily stand-ups, sprint reviews, and retrospectives, will be conducted to monitor progress and address challenges promptly. The use of Scrum allows for continuous feedback from stakeholders, early detection of issues, and incremental delivery of functional components of the system. This approach ensures that the results management system is developed efficiently while remaining adaptable to changing requirements.

2.1 Scrum Roles

The Scrum framework defines specific roles to ensure effective collaboration, accountability, and successful project delivery. In this project, the Scrum roles are adapted to support the development of the results management system.

Product Owner
The Product Owner is responsible for defining the system requirements and prioritizing the product backlog based on stakeholder needs. This role ensures that the system delivers maximum value by clearly communicating expectations and approving completed features.

Scrum Master
The Scrum Master facilitates the Scrum process and ensures that Scrum principles and practices are followed throughout the project. This role supports the development team by removing obstacles, organizing Scrum meetings, and promoting continuous improvement.

Development Team
The Development Team is responsible for designing, developing, testing, and implementing the system functionalities. The team works collaboratively during each sprint to deliver incremental and functional components of the results management system.

These roles collectively promote transparency, efficiency, and quality in the system development process while ensuring alignment with Scrum best practices.

2.2 System Backlog

System Backlog

The system backlog defines the core functional requirements of the results management system, categorized according to user roles to ensure clarity, accountability, and effective sprint planning. The backlog items are prioritized based on system importance and stakeholder needs and will be refined continuously throughout the Scrum development process.

Student Backlog

- ✓ Ability to log into the system using a registration number and secure credentials
- ✓ Ability to search and view academic results by selecting the appropriate semester
- ✓ Ability to view results in a clear and user-friendly format
- ✓ Ability to log out securely from the system

Lecturer Backlog

- ✓ Ability to log into the system using authorized lecturer credentials
- ✓ Ability to add students' academic results for assigned courses
- ✓ Ability to edit or update existing student results
- ✓ Ability to view and manage student result records
- ✓ Ability to log out securely from the system

Administrator (Dean) Backlog

- ✓ Ability to log into the system as a super administrator
- ✓ Ability to create, edit, and delete student and lecturer accounts
- ✓ Ability to manage and oversee all academic result records
- ✓ Ability to edit or delete results when necessary
- ✓ Ability to control system access and maintain data integrity

This structured backlog ensures that system functionalities are aligned with user responsibilities while supporting efficient and incremental development through Scrum.

2.3 Sprint Schedule

The sprint schedule outlines the planned timeline and structure for the development of the results management system using the Scrum framework. The project will be implemented through a series of short, time-boxed sprints, each focusing on the delivery of specific system functionalities. This approach enables incremental development, continuous testing, and regular review of progress to ensure that project objectives are achieved within the defined timeframe.

The project is scheduled to commence in the coming week and conclude on **April 19**, with each sprint assigned clear goals and deliverables aligned with system requirements and user roles.

Sprint	Sprint Duration	Sprint Title	Key Activities / Deliverables
Sprint 1	Mar 23 – Mar 29	System Planning and User Authentication	Requirements analysis, system architecture design, project setup, and implementation of authentication for students, lecturers, and administrator
Sprint 2	Mar 30 – Apr 4	Student Module Development	Development of student result search by registration number and semester, result display, and student user interface

Sprint 3	Apr 5 – Apr 9	Lecturer Module Development	Implementation of lecturer functionalities including adding, editing, and managing student results, along with validation
Sprint 4	Apr 10 – Apr 14	Administrator Module and System Integration	Development of administrative features for the dean, user management, result oversight, system integration, and bug fixes
Sprint 5	Apr 15 – Apr 19	System Testing and Deployment	Comprehensive system testing, documentation, final review, and system deployment

Table 1: Sprint Schedule 1

This sprint schedule ensures a structured yet flexible development process, allowing for timely delivery of a reliable, efficient, and user-centred results management system.

2.4 Scrum Board

A Scrum board is a visual tool used to track the progress of tasks during each sprint. It provides transparency, helps organize work, and allows the team to monitor the status of each backlog item in real-time. For this project, the Scrum board will display tasks related to the development of the results management system, categorized by user roles (students, lecturers, and administrator) and divided into columns representing their status: **To Do**, **In Progress**, and **Done**.

The Scrum board will be updated daily during stand-up meetings to reflect the current state of development and ensure accountability among team members. It also serves as a communication tool for stakeholders to observe project progress and identify any obstacles early.

Backlog Item / Task	Assigned To	Sprint	Status
Implement student login	Developer 1	Sprint 1	To Do
Design student result search UI	Developer 2	Sprint 2	To Do
Add results entry feature for lecturers	Developer 1	Sprint 3	To Do
Admin user management module	Developer 2	Sprint 4	To Do
System testing and documentation	Team	Sprint 5	To Do

Table 2: Scrum Board 1

This Scrum board allows the project team to manage tasks efficiently, prioritize work, and ensure timely delivery of functional components for each sprint. It also supports transparency and effective collaboration throughout the project lifecycle.

3. User and System Requirements

The requirements for the results management system are divided into **user requirements** and **system requirements** to clearly define the expectations, functionalities, and constraints. These requirements ensure that the system meets the needs of all stakeholders while providing a reliable, secure, and efficient platform for managing academic results.

3.1. User Requirements

User requirements define what each type of user should be able to do with the system:

Students

- ✓ Search and view their academic results by registration number and semester.
- ✓ Access the system using secure login credentials.
- ✓ View results in a clear and understandable format.

- ✓ Log out securely after completing their session.

Lecturers

- ✓ Log into the system using secure credentials.
- ✓ Add, edit, and update student results for assigned courses.
- ✓ View and manage student result records efficiently.
- ✓ Log out securely after completing their tasks.

Administrator (Dean)

- ✓ Access the system with super administrator privileges.
- ✓ Create, edit, and delete student and lecturer accounts.
- ✓ Manage and oversee all academic result records.
- ✓ Edit or delete results when necessary.
- ✓ Ensure data integrity and system security.

3.2. System Requirements

System requirements describe the technical functionalities and performance criteria the system must satisfy:

3.2.1. Functional Requirements

- ✓ User authentication and role-based access control.
- ✓ Student result search and retrieval by semester and registration number.
- ✓ Result management module for lecturers (add, edit, delete results).
- ✓ Administrative control for user management and result oversight.
- ✓ Secure storage of all academic data with backup and recovery mechanisms.

3.2.2. System Non-Functional Requirements

Category	Requirement
Performance	System should respond to search queries within 3 seconds
Security	Data must be protected using encryption and secure login protocols
Reliability	System uptime should be at least 99%
Usability	User interface should be intuitive and accessible for all users
Maintainability	System should allow for easy updates, bug fixes, and feature enhancements

Table 3: Non - Functional Requirement 1

These requirements ensure that the results management system is **user-centred, secure, and technically robust**, meeting both functional needs and performance expectations.

4. System Design

The system design for the results management system provides a detailed blueprint of how the system will function, interact with users, and manage data. The design focuses on modularity, security, and scalability to ensure an efficient and reliable system. It incorporates both **logical** and **physical design elements**, including architecture, data flow, and user interface considerations.

1. System Architecture

The system adopts a **three-tier architecture** consisting of:

1. **Presentation Layer (User Interface):**
    - Provides access for students, lecturers, and administrators.
    - Facilitates interaction with the system through forms, dashboards, and search interfaces.
2. **Business Logic Layer (Application Layer):**
    - Processes user requests, validates data, and enforces business rules.
    - Handles result calculations, updates, and user management.
3. **Data Layer (Database):**
    - Stores all academic records, user data, and system logs securely.
    - Ensures data integrity, backup, and recovery mechanisms.

2. Data Flow

The system's data flow ensures smooth communication between users and the database:

- ✓ Students submit search requests → System validates input → Results retrieved from database → Displayed to student.
- ✓ Lecturers input or update results → System validates and stores data → Confirmation provided.
- ✓ Administrator performs user management or system oversight → System updates database and logs all changes for accountability.

3. Database Design

The database design is structured to support **efficient storage, retrieval, and management of academic data**:

- ✓ **Students Table:** Stores registration numbers, personal details, and login credentials.
- ✓ **Lecturers Table:** Stores lecturer details and login credentials.
- ✓ **Results Table:** Stores course results linked to student and lecturer IDs.
- ✓ **Admin Table:** Stores dean credentials and logs administrative actions.
- ✓ Relationships are defined to maintain **referential integrity** between tables.

4. User Interface Design

The system interface is designed for **simplicity and clarity**:

- ✓ Students access results through a search form with minimal steps.
- ✓ Lecturers interact via dashboards for adding or updating results.
- ✓ Administrators manage users and records using a control panel with clearly labelled functions.

5. Security Design

- ✓ Role-based access ensures that each user type can access only authorized features.
- ✓ Data encryption and secure login protocols protect sensitive academic information.
- ✓ System logs all critical actions for auditing and accountability.

4.1. Use Case Diagram

The use case diagram provides a high-level view of the interactions between the users and the results management system. It identifies the system actors, their roles, and the functionalities they can access, ensuring clarity in system requirements and design.

Actors

1. **Student** – Searches and views academic results.
2. **Lecturer** – Adds, edits, and manages student results.
3. **Administrator (Dean)** – Oversees all system activities, manages users, and controls academic records.

Use Cases

- **Student:**
  - Login/Logout
  - Search Results by Registration Number and Semester
  - View Results
- **Lecturer:**
  - Login/Logout
  - Add Results
  - Edit/Update Results
  - View Student Records
- **Administrator (Dean):**
  - Login/Logout
  - Manage Users (Add/Edit/Delete Students and Lecturers)
  - Oversee Results
  - Edit/Delete Results
  - Maintain Data Integrity

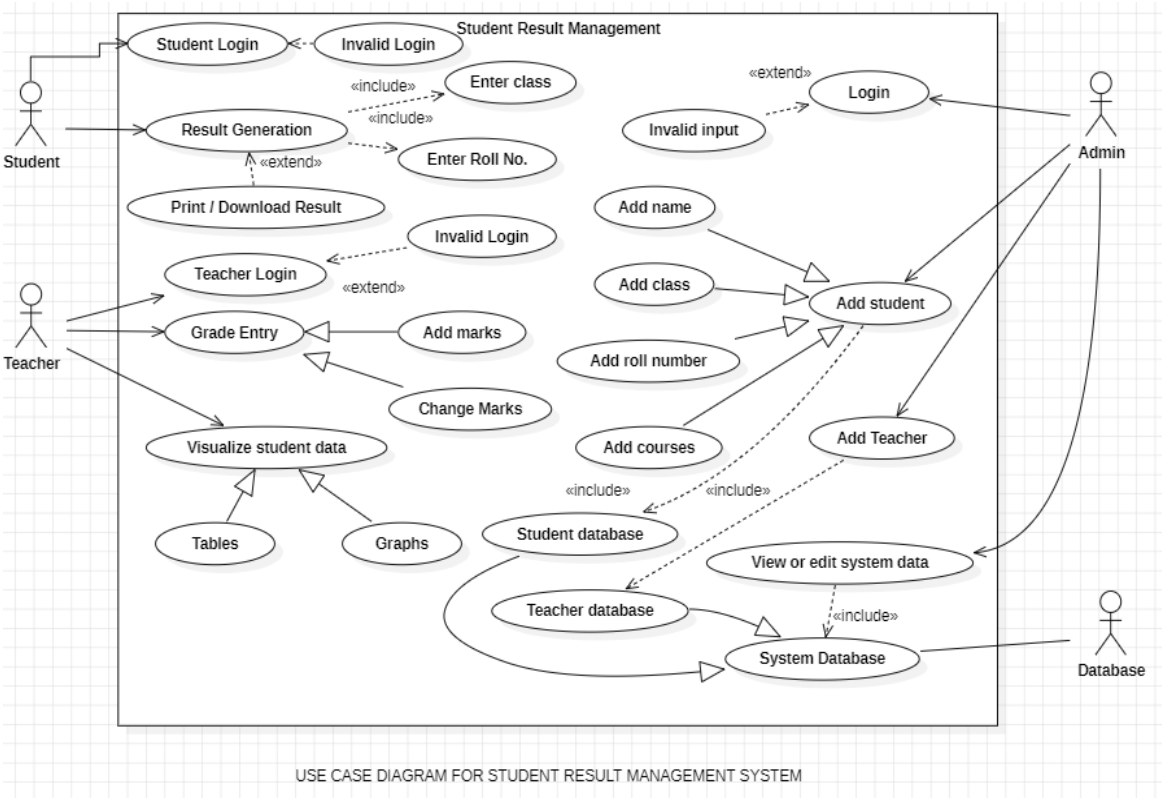


Figure 1: Use Case Diagram 1

The diagram highlights the interaction between the system and each actor, showing which functionalities are accessible to which user type. This ensures role-based access and helps developers understand the scope of system operations.

4.2 Class Diagram

A class diagram is a Unified Modelling Language (UML) diagram that represents the **static structure of a system**. It illustrates the system's classes, their attributes, methods, and the relationships between them. In this project, the class diagram defines the core entities of the results management system and shows how students, lecturers, administrators, results, and courses interact within the system.

Class Name	Attributes
Student	studentID, name, registrationNumber, semester, password
Lecturer	lecturerID, name, email, password
Administrator (Dean)	adminID, name, email, password
Result	resultID, studentID, courseCode, semester, score
Course	courseCode, courseName, lecturerID

Table 4: Class Attributes 1

Class Relationship	Relationship Type	Description
Student – Result	One-to-Many	One student can have multiple academic results
Lecturer – Result	One-to-Many	One lecturer can manage multiple student results
Course – Result	One-to-Many	One course can have multiple results for different students
Administrator – Student	One-to-Many	Administrator manages multiple student accounts
Administrator – Lecturer	One-to-Many	Administrator manages multiple lecturer accounts

Table 5: Class Relationships 1

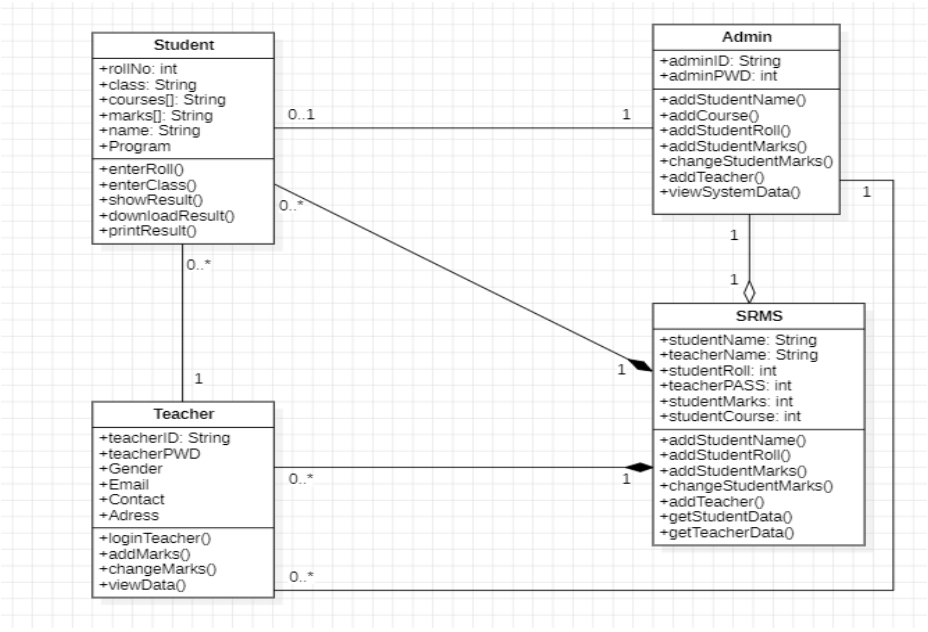


Figure 2: Class Diagram 1

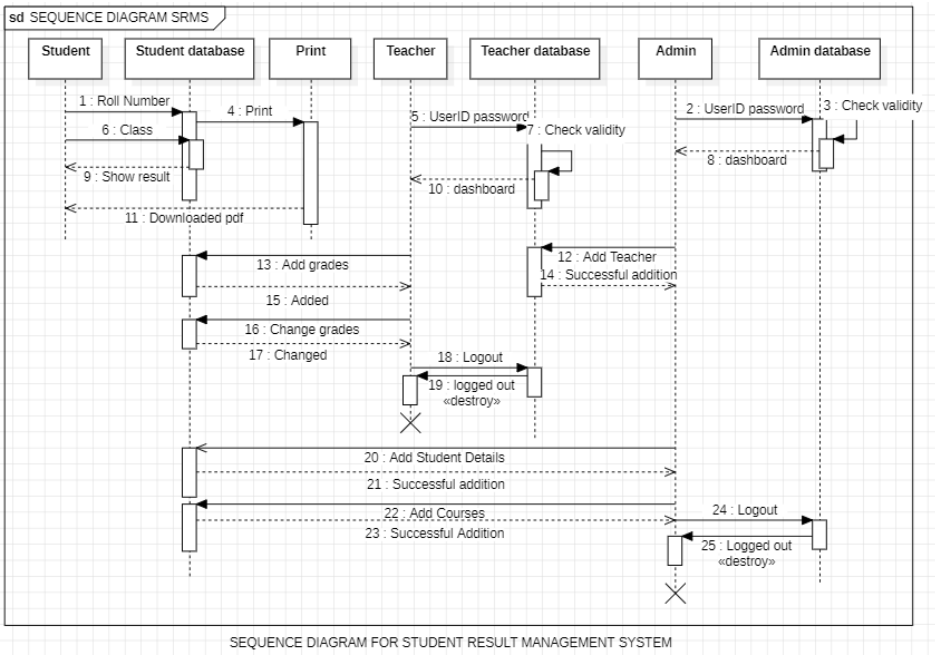


Figure 3: Sequence Diagram 1

4.3. Activity Diagram

An activity diagram is a UML behavioural diagram that illustrates the **workflow of activities** within a system. It shows the sequence of actions, decision points, and parallel processes involved in achieving a specific task. In this project, the activity diagram represents the flow of actions performed by different users while interacting with the results management system.

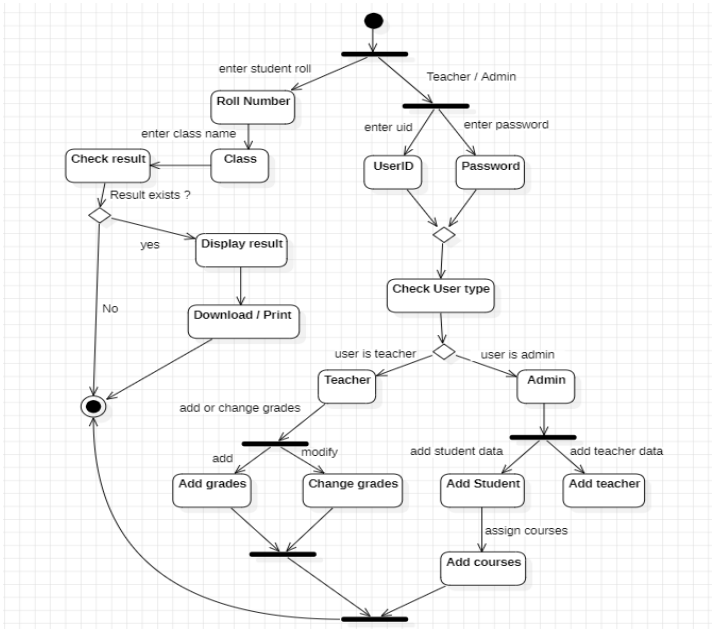


Figure 4: Activity Diagram 1

4.4. Database Design

The database design defines the structure used to store, organize, and manage data within the results management system. The design focuses on **data integrity, efficiency, and security**, ensuring accurate storage and fast retrieval of academic records. A relational database model is adopted to support structured data storage and enforce relationships between entities.

Table Name	Attributes
Students	studentID (PK), name, registrationNumber, semester, password
Lecturers	lecturerID (PK), name, email, password
Administrators	adminID (PK), name, email, password
Courses	courseCode (PK), courseName, lecturerID (FK)
Results	resultID (PK), studentID (FK), courseCode (FK), semester, score

Table 6: Database Tables and Attributes 1

Related Tables	Relationship Type	Description
Students – Results	One-to-Many	One student can have multiple academic results
Lecturers – Courses	One-to-Many	One lecturer can be assigned to multiple courses
Courses – Results	One-to-Many	One course can have multiple student results
Administrators – All Tables	Administrative Control	Administrator manages and oversees all database records

Table 7: Database Relationships 1

This database design supports efficient **result storage, retrieval, and management**, while maintaining data consistency and security.

4.5. Deployment Diagram

A deployment diagram is a UML diagram that illustrates the **physical deployment of system components** on hardware nodes and shows how software artifacts are distributed across the system environment. It helps in understanding how the results management system will be installed, executed, and accessed in a real-world setting.

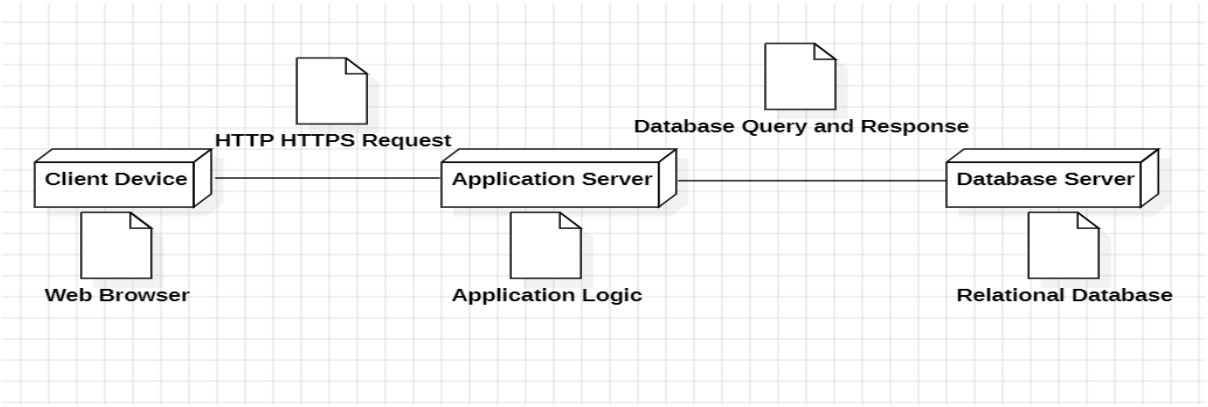


Figure 5: Deployment Diagram 1

This deployment diagram clearly demonstrates how the system components are **physically arranged and interact** to deliver a secure and efficient results management system.

The results management system follows a **client–server architecture**, where users access the system through client devices connected to a centralized server. The deployment structure consists of the following nodes:

- ✓ **Client Node:**
  Represents user devices such as computers or mobile devices used by students, lecturers, and the administrator to access the system through a web browser.

- ✓ **Application Server:** Hosts the system's application logic, user authentication services, and business rules. It processes requests from clients and communicates with the database server.
- ✓ **Database Server:** Stores all academic data, including student records, results, user credentials, and system logs. It ensures secure and reliable data storage.

Deployment Components and Nodes

Node	Deployed Components	Description
Client Device	Web Browser (UI)	Provides user interface for students, lecturers, and administrator
Application Server	Application Logic, Authentication Module	Handles system processing, validation, and business rules
Database Server	Relational Database	Stores and manages academic records and user data

- ✓ Client devices communicate with the application server over a secure network connection.
- ✓ The application server interacts with the database server to retrieve and store data.
- ✓ Security measures such as authentication and access control are enforced at the application server level.

5. Conclusion

This project proposes the design and development of a results management system aimed at improving the efficiency, accuracy, and accessibility of academic result processing. By leveraging the Scrum methodology, the project ensures an iterative and collaborative development process that supports timely delivery and continuous improvement. The system is designed to provide students with secure and convenient access to their academic results, enable lecturers to manage and update results efficiently, and empower the administrator (dean) with full control over user and result management. Through clearly defined system requirements, structured system design, and comprehensive UML diagrams, the project establishes a solid foundation for successful implementation. The proposed solution addresses key challenges associated with manual result handling and enhances data integrity, security, and usability. Upon completion, the system is expected to contribute significantly to improved academic administration and overall institutional effectiveness.